

Three ways to hand over a file

FTP, NFS and SMB, each on its own server, all reached from one client on the same wire. Same job, three answers — and three different ideas of who you are. From a running lab.

Command `qlab run filesharing-lab`

VMs `filesharing-lab-ftp · -nfs · -samba · -client`

LAN `192.168.100.0/24`, isolated between the four VMs

Credentials `labuser / labpass · lab accounts alice, bob / labpass`

Outcome `qlab test filesharing-lab` → 4 exercises, 52 checks, all passed

1. Four machines, one wire

filesharing-lab-ftp	192.168.100.1/24
filesharing-lab-nfs	192.168.100.2/24
filesharing-lab-samba	192.168.100.3/24
filesharing-lab-client	192.168.100.10/24

Three servers, one protocol each, and a client that talks to all of them. The addresses are fixed, so nothing in this walkthrough depends on what DHCP felt like doing. The client is the interesting machine: everything after this section is the same task — read a file that lives somewhere else — carried out three times from there.

Splitting the servers apart is the point. On one box the three protocols blur together; on three boxes each one has to bring its own transport, its own authentication and its own idea of what a "file" is.

2. What each server is actually running

```
--- ftp (vsftpd) ---
active
anonymous_enable=YES
anon_root=/srv/ftp/shared
local_enable=YES
write_enable=YES
chroot_local_user=YES
pasv_enable=YES
--- nfs (NFS-Ganesha: an NFS server that runs in userspace) ---
active
    Protocols = 3, 4;
    Path = /srv/nfs/shared;
    Pseudo = /shared;
    Access_Type = RW;
    Squash = No_Root_Squash;
        Clients = 192.168.100.0/24;
        Access_Type = RW;
    Path = /srv/nfs/readonly;
    Pseudo = /readonly;
    Access_Type = RO;
    Squash = Root_Squash;
        Clients = 192.168.100.0/24;
        Access_Type = RO;
--- samba (smbd) ---
active
[global]
[shared]
    path = /srv/samba/shared
    read only = No
    valid users = alice bob
[public]
    guest ok = Yes
    path = /srv/samba/public
```

Three daemons, three configuration styles.

vsftpd is configured by flat `key=value` lines. Two of them matter here: `anonymous_enable` opens a login with no account behind it, rooted at `anon_root`, and `chroot_local_user` locks a real account into its own home. One daemon, two quite different experiences depending on the name you type.

The NFS server is **NFS-Ganesha**, which runs in userspace rather than in the kernel — worth noticing, because most NFS documentation assumes `nfs-kernel-server` and `/etc/exports`, and neither exists on this machine. Each export names a real `Path`, a `Pseudo` name clients use instead, an `Access_Type` and a `Squash` policy. The two exports differ in exactly those last two fields.

Samba answers `testparm` with the config it has actually parsed. `shared` has `valid users`, `public` has `guest ok` — the same distinction `vsftpd` draws with `anonymous_enable`, written in a different dialect.

3. FTP: a conversation with numbers in front

```
$ a whole FTP session, scripted – anonymous first
Connected to 192.168.100.1.
220 (vsFTPd 3.0.5)
230 Login successful.
Remote system type is UNIX.
229 Entering Extended Passive Mode (|||30037|)
150 Here comes the directory listing.
-r-xr-xr-x   1 109      117          43 Sep 14 13:22 README.txt
-r-xr-xr-x   1 109      117          46 Sep 14 13:22 sample.txt
226 Directory send OK.
229 Entering Extended Passive Mode (|||30061|)
150 Opening BINARY mode data connection for README.txt (43 bytes).
226 Transfer complete.
221 Goodbye.

$ and the same server as a real account
220 (vsFTPd 3.0.5)
230 Login successful.
Remote system type is UNIX.
Remote directory: /
221 Goodbye.

-rw-rw-r-- 1 labuser labuser 43 Sep 14 13:22 /tmp/from-ftp.txt
Welcome to the filesharing-lab FTP server.
```

FTP is old enough to be readable. Every server reply is a three-digit code and a sentence: `220` on connect, `230` for a successful login, `150` before data starts moving, `226` when it is done, `221` on the way out. That is the whole protocol, and it is why FTP is the protocol people learn to debug by hand with `telnet`.

Two logins run here against the same daemon. The anonymous one lands on the directory `anon_root` points at and sees the two files placed there. Then `alice` logs in with a real password and `pwd` answers `/` — not because she is at the root of the server, but because `chroot_local_user` has made her home *look* like the root. She cannot see the anonymous directory, or anything else on the machine.

The password went across in clear text. So does everything in the listing. FTP predates the assumption that the network is hostile; on a real network this is FTPS or SFTP, and the reason the lab keeps plain FTP is that you can read it.

4. NFS: the file server that disappears

```
$ showmount -e 192.168.100.2      (what the server offers)
Export list for 192.168.100.2:
/srv/nfs/shared    192.168.100.0/24
/srv/nfs/readonly  192.168.100.0/24

$ mount -t nfs, then look at what landed in the filesystem
192.168.100.2:/shared on /mnt/nfs type nfs4
(rw,relatime,vers=4.2,rsize=1048576,wsiz=1048576,namlen=255,hard,proto=tcp,timeo=600,retrans=2,sec=sys,
clientaddr=192.168.100.10,local_lock=none,addr=192.168.100.2)

total 4
-rw-r--r-- 1 alice alice 37 Sep 14 13:22 README.txt

$ writing through the mount
-rw-r--r-- 1 root root 36 Sep 14 13:35 /mnt/nfs/from-client.txt
```

Nothing here looks like a network protocol, and that is the whole idea. After `mount`, the remote directory is a directory. `ls` works. Writing works. No client program, no session, no transfer commands — the kernel turns ordinary filesystem calls into NFS requests and back.

`showmount -e` asks the server what it offers before committing to anything, and answers with the real paths. The mount itself uses the **pseudo** path instead: NFSv4 hangs every export off a single virtual root, so clients ask for `/shared` and never learn where it lives on the server's disk.

The `mount` line is worth reading once. `vers=4.2` is the negotiated protocol version, `hard` means a request retries forever instead of failing when the server is briefly unreachable, and `sec=sys` is the authentication — of which more in a moment.

5. `sec=sys` : identity is a number you send

```
NFS moves numbers, not names. The same file, from both ends:
--- on the server ---
total 8
-rw-r--r-- 1 alice alice 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 root  root  36 Sep 14 13:35 from-client.txt

total 8
-rw-r--r-- 1 2001 2001 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 0    0    36 Sep 14 13:35 from-client.txt
uid=2001(alice) gid=2001(alice) groups=2001(alice)
--- on the client ---
total 8
-rw-r--r-- 1 2001 2001 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 0    0    36 Sep 14 13:35 from-client.txt
```

```
total 8
-rw-r--r-- 1 alice alice 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 root  root  36 Sep 14 13:35 from-client.txt
uid=2001(alice) gid=2001(alice) groups=2001(alice)
```

The same two files, listed on both machines, first as numbers and then as names.

The numbers cross the wire; the names never do. `alice` is uid 2001 on the server, and this lab gives the client an `alice` at uid 2001 as well, so both sides print the same name and everything looks natural. Change the client's `alice` to 1001 and the file would show up owned by whoever *is* 2001 there — or by a bare `2001` if nobody is. Nothing would be broken, and nothing would warn you.

`from-client.txt` is the other half of the lesson. It was written by `sudo` on the client and it belongs to `root` on the server, because this export is configured `Squash = No_Root_Squash`: the server takes uid 0 at its word. The default — and what the read-only export uses — is `Root_Squash`, which maps a remote root down to `nobody` precisely so that administrative access on a client does not become administrative access on the server's files.

`sec=sys` means the server trusts the client's word about who is asking. That is an entirely reasonable model on a machine-room network you control, and it is why NFS is normally kept off any network you do not. The alternative, `sec=krb5`, puts Kerberos in the path so identity is proven rather than asserted.

6. SMB: shares, not filesystems

```
$ smbclient -L      (what the server publishes)

  Sharename      Type      Comment
  -----
  shared         Disk      Authenticated Read-Write Share
  public         Disk      Public Read-Only Share
  IPC$           IPC       IPC Service (filesharing-lab Samba Server)
SMB1 disabled -- no workgroup available

$ fetch a file over SMB
getting file \README.txt of size 39 as from-smb.txt (390000.0 KiloBytes/sec) (average inf KiloBytes/sec)
.                D          0   Mon Sep 14 13:22:35 2026
..               D          0   Mon Sep 14 13:22:35 2026
README.txt       N          39  Mon Sep 14 13:22:35 2026

      2051980 blocks of size 1024. 465716 blocks available
-rw-r--r-- 1 labuser labuser 39 Sep 14 13:35 /tmp/from-smb.txt

$ the same share without credentials
tree connect failed: NT_STATUS_ACCESS_DENIED
```

SMB sits between the other two. It is a session protocol like FTP — you connect, authenticate, and run commands — but what it exposes is a *share*, a named tree that can be mounted like NFS or browsed like FTP. `smbclient` here is the browse-it-by-hand version.

The share list is the server's public catalogue: the two configured shares plus `IPC$`, the control channel SMB uses for the share list itself. Then `get` pulls a file. The "SMB1 disabled" line is not

an error — it is `smbclient` noting that the ancient workgroup-browsing protocol is switched off, as it should be.

The last command is the one that matters: the same share, no credentials, `NT_STATUS_ACCESS_DENIED`. Authorisation is per-share and checked by the server, which is the structural difference from NFS. Here the server decides; with `sec=sys` NFS, the client does.

7. The three side by side

Same job, three protocols, seen from one machine:

PROTO	PORT	HOW IT APPEARS	IDENTITY
FTP	21 + data	a session; you copy files in and out	username + password, in clear
NFS	2049	a directory in your filesystem	numeric uid/gid, trusted
SMB	445	a share you mount or browse	username + password, per share

```
$ what is actually mounted on this client right now
192.168.100.2:/shared on /mnt/nfs type nfs4 (rw, relatime, vers=4.2, rsize=1048576, wsize=1048576, n
```

```
$ and the three servers, from the client
192.168.100.1:21 open
192.168.100.2:2049 open
192.168.100.3:445 open
```

The three servers are reachable on their three ports, and exactly one of the three protocols shows up in `mount` — because only NFS was designed to become part of the local filesystem. FTP and SMB can be made to look like that too (`curlftps`, `mount -t cifs`), but for FTP that is a client-side illusion, and for SMB it is a second way of using a protocol that already had a session.

The useful summary is the identity column. FTP names a user per session, in clear. SMB names a user per share, checked by the server. NFS names nobody at all: it sends a number and trusts you. Three protocols, three answers to the only question that really separates them.

8. Try it yourself

```
qlab run filesharing-lab
qlab shell filesharing-lab-client
```

From the client:

```
ftp -inv 192.168.100.1 # anonymous / alice+labpass
sudo mount -t nfs 192.168.100.2:/shared /mnt/nfs-shared
sudo mount -t nfs 192.168.100.2:/readonly /mnt/nfs-readonly
smbclient //192.168.100.3/shared -U alice%labpass -m SMB3
smbclient //192.168.100.3/public -N -m SMB3
```

Three things worth doing after that:

- Write to `/mnt/nfs-readonly` and watch which layer refuses you.

- Change `alice`'s uid on the client, remount, and see the ownership of `README.txt` change without a single error message.
- Run `sudo tail -f /var/log/vsftpd.log` on the FTP server while you log in from the client — the whole conversation, both sides, in plain text.

```
Exercise 1 – FTP with vsftpd
  Exercise 1: All 11 checks passed
Exercise 2 – NFS with Ganesha
  Exercise 2: All 12 checks passed
Exercise 3 – Samba
  Exercise 3: All 14 checks passed
Exercise 4 – Cross-Protocol Comparison
  Exercise 4: All 15 checks passed
All 4 exercise(s) passed!
```

`qlab test filesharing-lab` runs 52 checks across the four VMs. The last exercise is the cross-protocol one, and its `Alice UID matches (2001)` check is there for the reason section 5 explains: on this lab the number agrees, and the test says so out loud rather than letting it look like magic.